ATOM User Guide

Version 6.2.0, October 2026

José Luís Martins
Departamento de Física, Instituto Superior Técnico, Lisboa, Portugal
INESC MN, Lisboa, Portugal
jlmartins@inesc-mn.pt
jose.l.martins@tecnico.ulisboa.pt

Section 4 adapted from the ATOM documentation of SIESTA by
Alberto García, Universidad del País Vasco, Bilbao, Spain

Contents

1 Introduction **2**

2 Installation **2**

2.1 Step 1: Downloading and extracting the archive 2

2.2 Step 2: Generating the documentation 3

2.3 Step 3: Compiling the code 3

3 Running the code **4**

3.1 Running all of it at once 4

3.1.1 First command line argument 4

3.1.2 Second and third command line arguments 5

3.2 The all-electron structure 5

3.3 Constructing the pseudopotential 5

3.4 Visualizing the pseudopotential 6

3.5 Applying the Kleinman-Bylander transformation. 6

3.6 Visualizing the KB operators. 6

3.7 Logarithmic derivatives plots. 7

3.8 Testing the pseudopotentials 7

4 The atom.dat input file **7**

4.1 All-electron calculations 7

4.2 Pseudopotential generation 8

4.3 Core corrections 9

4.4 Format of the file 10

5 A primer on ab-initio pseudopotentials **13**

1 Introduction

This is a modernized version of a old code that generates and tests pseudopotentials to be used in electronic structure theory calculations.

The original code was written by Sverre Froyen at the University of California Berkeley. Phys. Rev. B 26, 1738 (1982), https://doi.org/10.1103/PhysRevB.26.1738

That code had major enhancements by Norman J. Troullier at the University of Minnesota, Minneapolis. Phys. Rev. B 43, 1993 (1991), https://doi.org/10.1103/PhysRevB.43.1993

Since then the code has been maintained and improved by José Luís Martins. Peter Schuster and Manuel Maria Alemany wrote new plotting packages. GGA were introduced by Carlos Balbas and José Soler in a collaboration with the SIESTA code.

The present code is a "translation" from old FORTRAN (Fortran 70) to modern Fortran (90 → present) with more detailed documentation by José Luís Martins, who added a few enhancements. In `Doc/Historic` one can find the almost original code, that still compiles and runs!

A version used by the SIESTA code (`https://siesta-project.org/siesta/`) is maintained by Alberto García, who added some features and made substantial structural changes to the April 1990 (5.0) Minnesota version. Both versions can use the same input file as the original 1982 code. The description of that file, Section 4, was written mostly by Alberto García for the SIESTA documentation (version 3.2.2, January 2006) and adapted to the present code.

The program's basic capabilities are:

- All-electron DFT atomic calculations for arbitrary electronic configurations and a few Local Density Approximation (LDA) and Generalized Gradient Approximations (GGA) functionals.
- Generation of ab-initio pseudopotentials, and their transformation to the Kleinman-Bylander form with atomic basis functions.
- Atomic calculations in which the effect of the core is represented by a previously generated pseudopotential. These are useful to make sure that the pseudopotential correctly reproduces the all-electron results for the valence complex.

2 Installation

2.1 Step 1: Downloading and extracting the archive

The code is available from José Luís Martins home page:

`https://fenix.tecnico.ulisboa.pt/homepage/ist13146/pseudopotential`

and from GitHub

`https://github.com/jlm785/`

If you downloaded the `atom-6.x.y.tgz` file (where `x` and `y` are major and minor version numbers) just extract it,

```
$ tar xzf atom-6.x.y.tgz
```

and you will have a `atom-6.x.y` directory

If you cloned the git you already have the relevant code in the main directory.

2.2 Step 2: Generating the documentation

The documentation is in the `atom-6.x.y/Doc` directory. It includes this file, `user_guide.tex`, with a description of the key input file `atom.dat` in Section 4; a setup to generate a detailed description of the whole code for developers in the `Doc/doxy`.

For that detailed description you need doxygen (`https://www.doxygen.nl`) and graphviz (`https://graphviz.org/`), both are available on most distributions. To check that your computer has doxygen and graphviz installed run the commands

```
atom-6.x.y/Doc$ dpkg -s doxygen
```

```
atom-6.x.y/Doc$ dpkg -s graphviz
```

on a Debian based distribution (Ubuntu et al), or use equivalent tools (yum, rpm, dnf zypper,...) or the relevant GUI.

If they are not available just install them

```
atom-6.x.y/Doc$ sudo apt install doxygen
```

```
atom-6.x.y/Doc$ sudo apt install graphviz
```

again for Debian based distros, or use equivalent tools (yum, rpm, dnf zypper,...) or the relevant GUI.

Finally run doxyfile in the doxy directory

```
atom-6.x.y/Doc/doxy$ doxygen Doxyfile
```

If you know what you are doing you can edit `atom-6.x.y/Doc/doxy/Doxyfile`.

Finally it is useful to create a link to the file `atom-6.x.y/Doc/html/index.html`

```
atom-6.x.y/Doc$ ln -s html/index.html documentation.html
```

Opening that link in your browser will allow you to see the documentation for every file in the present code.

2.3 Step 3: Compiling the code

The code has been tested with several Fortran compilers, `ifort` (from Intel oneAPI), `gfortran` (from gnu), `pgfortran` (from Portland), and even LLVM (experimental Intel compiler).

As the performance is not an issue for these "simple" one-dimensional (radial Schrödinger) calculations just use the compiler that is more familiar for you.

The choice of compiler is in the `atom-6.x.y/Src/make.inc` file. You may edit it for your convenience, but in principle you have to make just two choices. The first is to identify which CPU you are using, as it narrows the choice of possible compilers. This is done by commenting/uncommenting the lines below `# compiler for the job`.

The second is the compiler you want, and is the key decision. Again comment/uncomment the lines below `# Suggestions for compilers`.

For any other modification of the file it is assumed you know what you are doing...

To compile just go to `atom-6.x.y/Src` and type make

```
atom-6.x.y$ cd Src
```

```
atom-6.x.y/Src$ make
```

In a few seconds you should have the main library `libatom_`*compiler*`.a` and a few executables

`atom_`*compiler*`.exe`, etc... where *compiler* is the name of the chosen compiler. As you probably are not interested in comparing compilers, modify the script `copyit` to simplify the names and relocate the executables. In my case I just copy them to `atom-6.x.y` removing the compiler name.

The code relies on gnuplot (`http://www.gnuplot.info`) to produce graphics. Just install it on your machine (apt, yum, rpm, dnf zypper,...) if you want to see them. Otherwise you will have to adapt the code to your favorite function plotter. `gnuplot` is simple and old, just send me your modifications for inclusion in a later release.

3 Running the code

I assume you have a working directory separate from the source. In the following I will call it `atom-6.x.y/WORK`

This is a code with a long history. Keeping it back-compatible (the original code still runs fine!) makes it easier to check that modifications did not introduce bugs, at the cost of some compromises, mainly in the number and format of files.

The old code and new additions communicated through files. The new code still uses the same file formats of several decades ago. The good news about the present version, is that except for their presence in the working directory you do not have to worry about them in most cases.

You can run the whole pseudopotential generation and testing procedure at once, or run step by step. As the code uses intermediary files you can go back and repeat the steps you want, for example to modify the default values of parameters or generate new graphics.

3.1 Running all of it at once

Assuming you have gnuplot installed and you copied the executables to the code root, you can run the whole pseudopotential generation and testing with a simple command:

```
atom-6.x.y/WORK$ ../atom_all.exe
```

the code will check if there is a `atom.dat` file in the working directory. This is the original input file of the old code and its strictly formatted structure is described in Section 4. If the file is present it will use it, otherwise the program will ask for the chemical symbol of the element you want, write a `atom.dat` file with default values. In both cases it will then proceed with the pseudopotential generation and testing.

After running the code you will find many files on your working directory. These are described in more detail in the next subsections. Briefly, you get the "input" file `atom.dat` and the detailed output file `atom.out`. The most important files are those that are read by other codes, `*_LDA__TM.UPF` for quantum-espresso, `*_POTKB_F.DAT` for cpw2000, `pseudo.dat` for SIESTA, and `psd.pot` for PARSEC. Finally you get many graphics related files, `*.pdf` and `*.gp`, and a few `*.dat` internal data files.

3.1.1 First command line argument

You can pass arguments in the command line, for example

```
atom-6.x.y/WORK$ ../atom_all.exe -h
```

will print a short help text and stop.

You can also run in a full interactive mode

```
atom-6.x.y/WORK$ ../atom_all.exe -x
```

where you are asked to override the default values, and therefore can be cumbersome. Overriding the defaults is easier by running only the relevant sections of the code.

The most useful first argument is the chemical symbol,

```
atom-6.x.y/WORK$ ../atom_all.exe Si
```

where you can swap "Si" for the chemical symbol of any element that interests you from Hydrogen to Oganesson (https://doi.org/10.1103/PhysRevB.26.4199). All the work will be done with default values without prompting the user.

3.1.2 Second and third command line arguments

Default values for the reference atomic configuration and pseudopotential radii can be overridden by careful editing the `atom.dat` file. Although the choice of using the partial core-correction for exchange and correlation can be altered in that way, it is easier to do it by the second argument of the command line, that takes the values `yes/no` force using or not using the partial core correction.

The code generates "SIESTA-like" atomic orbitals. The third argument that can take the values `SZ/DZ/DZP` for single-zeta, double zeta or double-zeta with polarization, indicates the quality of atomic orbitals that are included in the `*_POTKB_F.DAT` file.

3.2 The all-electron structure

The starting point of a pseudopotential construction is an all-electron calculation. This is done by running the "atom" code:

```
atom-6.x.y/WORK$ ../atom.exe
```

the program searches for a file `atom.dat` in the `WORK` directory with the information on the atomic configuration and options of the calculation such as the exchange and correlation model. This is the key input file and is described in Section 4.

If the file `atom.dat` is not present, the code asks the user the chemical symbol of the element to be calculated, writes a `atom.dat` file with the default values and runs the calculation. The user may edit the `atom.dat` file to change the default choices, taking into account that the file is for fixed format read (back-compatibility).

In `atom-6.x.y/Doc/atom_table.dat` the user can find an example of input file for elements H to Pu.

At the end of the calculation a file `datafile.dat` is written with the information of the atomic configuration and self-consistent potential.

3.3 Constructing the pseudopotential

The key operation is the construction of the pseudopotential. To generate the Troullier-Martins pseudopotential run

```
atom-6.x.y/WORK$ ../psd_gen.exe
```

The program reads the all-electron results from the `datafile.dat`, suggested core radii from

the `atom.dat` file, and presents that information plus data on maxima and last node of the valence wave-functions to the user and ask if it accepts the defaults. In the negative case the user chooses new core radii.

At the end of the calculation the user has three new files (back compatibility), an unformatted `pseudo.dat` file used later in tests and compatible with SIESTA, a `plot.dat` file used for later plotting, and a formatted `psd.pot` file compatible with parsec and used for the Kleinman-Bylander transformation. This last file is therefore the key output file of the program.

3.4 Visualizing the pseudopotential

If `gnuplot` is installed the user can compare pseudo and all-electron wave-functions, observe the pseudopotential and respective Fourier transforms by running

```
atom-6.x.y/WORK$ ../plot_show.exe
```

The program will read `plot.dat` (or another file chosen by the user) and allow the user to change the range of the axis of the plots, and save a `pdf` version of the plots. If the user doesn't clean the `*.gp` files at the last question, the `WORK` directory will have several `command_`*name*`.gp` and *name*`.gp` files. The user can see the plots again with gnuplot

```
atom-6.x.y/WORK$ command_name.gp
```

and can edit the command file to change the plot. The user can also use the data in *name*`.gp` files to generate plots with other software, including xmgrace.

3.5 Applying the Kleinman-Bylander transformation.

In plane-wave codes the pseudopotential is used in its Kleinman-Bylander form. That transformation plus the generation of an efficient atomic basis set can be achieved by

```
atom-6.x.y/WORK$ ../kb_conv.exe
```

the program will read the `psd.pot` file and ask the users if he accepts the defaults and in the negative case allows the choice of new parameters.

The key decision here is the choice of the local component of the pseudopotential. A bad choice can introduce "ghost states" and the code will check if they appear.

At the end of the calculation the program writes three new files (back compatibility), an unformatted `pseudo_kb.dat` used later in tests, a `plot_kb.dat` file used for later plotting, and a formatted *Xy*`_POTKB_F.DAT` file compatible with the `cpw2000 code` where *Xy* is the chemical symbol of the relevant element.

The program also suggests the range of energy cutoffs for the plane-wave calculations. It is a range because different properties converge at different rates.

3.6 Visualizing the KB operators.

If `gnuplot` is installed the user can compare the pseudo wave-functions with the atomic basis functions, observe the KB operators and respective Fourier transforms, and have an estimate of the rate of convergence of the total energy with plane-wave basis size by running

```
atom-6.x.y/WORK$ ../plot_kb_show.exe
```

The program will read the `plot_kb.dat` file or another file specified by the user. See the section

on visualizing the pseudopotential for dealing with the files generated by the program.

3.7 Logarithmic derivatives plots.

If `gnuplot` is installed the user can generate the plots of the logarithmic derivatives of the wave-functions with respect to energy for a given radius with

```
atom-6.x.y/WORK$ ../plot_ln_show.exe
```

The program will read the `pseudo.dat`, `pseudo_kb.dat` and `datafile.dat` files (or the files specified by the user). See the section on visualizing the pseudopotential for dealing with the files generated by the program.

3.8 Testing the pseudopotentials

By editing the `atom.dat` file and running

```
atom-6.x.y/WORK$ ../atom.exe
```

the user can either see the all electron results for a different configuration (`ae` keyword on the first line and deleting the second line, back-compatibility as you should have guessed) or see the result of a different configuration on the pseudopotential described in `pseudo.dat`. Running

```
atom-6.x.y/WORK$ ../atom_kb.exe
```

the user achieves the same type of test for the KB pseudopotential described by `pseudo_kb.dat`.

A series of tests of ionization and promotion energies can be done with the original `atom.dat` by running

```
atom-6.x.y/WORK$ ../atom_test.exe
```

At the end the program will print a comparison of the ionization energies and promotion energies with all- electrons pseudopotential and pseudopotential in KB form.

Discrepancies on the range of a few hundredth of eV are to be expected, anything larger may indicate a bad pseudopotential.

4 The atom.dat input file

4.1 All-electron calculations

Assume we want to find the orbital eigenvalues, total energy, and/or charge density of Si in its ground state for some functional. You already have the compiled executables from SIESTA or from the ATOM code that you can find in the `Src/` folder. You should now go to a "working" directory and try the following: Our input file is named `atom.dat` and contains the lines (see Sect. 4.4 for more details):

```
   ae     Si   ground state all-electron
 n=Si c=ca
      14.0       0.0       0.0     120.0       6.0      80.0
    3    2
    3    0     2.00      0.00
    3    1     2.00      0.00
```

```
#23456789012345678901234567890123456789012345678901234567890      Ruler
```

We can run the calculation by typing *pathtoexecutables*`/atom.exe > atom.out` or *pathtoexecutables*`/atom_`*compilername*`.exe > atom.out` Here we are redirecting the output to `atom.out`. As described in Section 2.3, the present release of the code adds the compiler name by default, for example `atom_gfortran.exe` for code compiled with `gfortran`.

It is interesting to peruse the `atom.out` file. In particular, it lists the orbital eigenvalues (in Rydbergs, as almost every other energy in the program):

```
 nl     s      occ         eigenvalue     kinetic energy      pot energy

 1s    0.0    2.0000    -130.36911227      183.01377610    -378.73491457
 2s    0.0    2.0000     -10.14892691       25.89954256     -71.62102165
 2p    0.0    6.0000      -7.02876265       24.42537869     -68.74331196
 3s    0.0    2.0000      -0.79662738        3.23745216     -17.68692614
 3p    0.0    2.0000      -0.30705175        2.06135786     -13.62572529
```

(For a relativistic or spin-polarized calculation, there would be "up" and "down" flags in the `s` column above.)

4.2 Pseudopotential generation

To generate a pseudopotential for Si, using the Troullier-Martins scheme repeat the calculation with relativistic effects (although they are quite small) and use the LDA (Ceperley-Alder flavor). The input file contains the lines (see Sect. 4.4 for more details):

```
   pg     Si     Silicon
        tm2
 n=Si c=car
      14.0       0.0       0.0     120.0       6.0      80.0
    3    4
    3    0     2.00      0.00
    3    1     2.00      0.00
    3    2     0.00      0.00
    4    3     0.00      0.00
    1.90     1.90     1.90     1.90

#23456789012345678901234567890123456789012345678901234567890      Ruler
```

Note the two extra lines with respect to an all-electron calculation. The pseudopotential core radii for all channels are 1.90 bohrs. Even though they are nominally empty in the ground state, we include the $3d$ and $4f$ states in order to generate the corresponding pseudopotentials.

It is certainly cumbersome to type a fixed format input file. The good news is that running `/atom_`*compilername*`.exe` in a folder without the `atom.dat` and entering the chemical symbol of the element will generate such a file with default values, which can be modified for subsequent runs. Notice that the default values may not be the best, so that file is just a template to be edited!

We can run the calculation by typing again *pathtoexecutables*/`atom.exe` > `atom.out` or *pathtoexecutables*/`atom_`*compilername*`.exe` > `atom.out` The `atom.out` file now has the spin-orbit effect and we see in the effect on eigenvalues that is small. The empty scattering orbitals may have a small positive eigenvalue.

nl	s	occ	eigenvalue	kinetic energy	pot energy
1s	0.5	2.0000	-130.51541377	0.00000000	-380.32924747
2s	0.5	2.0000	-10.18537055	0.00000000	-71.99156489
2p	-0.5	2.0000	-7.05518951	0.00000000	-69.08320077
2p	0.5	4.0000	-7.00775181	0.00000000	-68.71538886
3s	0.5	2.0000	-0.79937163	0.00000000	-17.74263422
3p	-0.5	0.6667	-0.30807126	0.00000000	-13.66178946
3p	0.5	1.3333	-0.30567131	0.00000000	-13.60785806
3d	-0.5	0.0000	0.00231324	0.00000000	-0.38199583
3d	0.5	0.0000	0.00231324	0.00000000	-0.38199582
4f	-0.5	0.0000	0.00340155	0.00000000	-0.35430233
4f	0.5	0.0000	0.00340155	0.00000000	-0.35430233

Running the pseudopotential generation by typing *pathtoexecutables*/`psd_gen.exe` > `psd.out` or *pathtoexecutables*/`psd_gen_`*compilername*`.exe` > `psd.out` will give you (just accept the defaults) a new default output where you can find the pseudopotential eigenvalues.

nl	s	occ	eigenvalue	kinetic energy	pot energy
1s	0.5	2.0000	-0.79936317	0.50555149	-3.74109919
2p	-0.5	0.6667	-0.30805322	0.77243757	-3.26354801
2p	0.5	1.3333	-0.30566087	0.76702417	-3.25195654
3d	-0.5	0.0000	0.00231324	0.00233363	-0.10914167
3d	0.5	0.0000	0.00231324	0.00233363	-0.10914167
4f	-0.5	0.0000	0.00340158	0.00342993	-0.10122710
4f	0.5	0.0000	0.00340158	0.00342993	-0.10122710

which are almost identical to the all-electron values.

4.3 Core corrections

The program can generate pseudopotentials with the non-linear exchange-correlation correction proposed in S. G. Louie, S. Froyen, and M. L. Cohen, Phys. Rev. B 26, 1738 (1982).

In the traditional approach (which is the default for LDA calculations), the pseudocore charge density equals the charge density outside a given radius r_{pc} had the smooth form

$$\rho_{pc}(r) = Ar\sin(br)$$

inside that radius. A smooth matching was provided with suitable A and b parameters calculated by the program.

A new scheme has been implemented to fix some problems in the generation of GGA pseudopotentials. The smooth function is now

$$\rho_{pc}(r) = r^2 \exp\left(a + br^2 + cr^4\right)$$

and derivatives up to the second are continuous at r_{pc}.

To use core corrections in the pseudopotential generation the jobcode in the first line should be `pe` instead of `pg`.

The radius r_{pc} should be given in the sixth slot in the last input line (see above). If it is negative or zero (or blank), the radius is then computed using the fifth number in that line (`rcore_flag`, see the example input file above) and the following criterion: at r_{pc} the core charge density equals `rcore_flag`*(valence charge density). It is *highly recommended* to set an explicit value for the pseudocore radius r_{pc}, rather than letting the program provide a default.

If `rcore_flag` is input as negative, the full core charge is used. If `rcore_flag` is input as zero, it is set equal to one, which will be thus the default if `pe` is given but no numbers are given for these two variables.

The seventh number in that line plays the role of `rcore_flag` for the partial core kinetic energy density. If it is zero or blank it is set equal to one.

The output file contains the radius used and the A (a) and b (and c) parameters used for the matching.

4.4 Format of the file

For historical reasons, the input file is in a rigid column format. Fortunately, most of the column fields line up, so the possibility of errors is reduced. We will begin by describing in detail a very simple input file for an **all-electron calculation** for the ground state of Si.

The file for Si could be:

```
   ae     Si   ground state all-electron
 n=Si c=ca
       0.0       0.0       0.0       0.0       0.0       0.0
    3    2
    3    0     2.00      0.00
    3    1     2.00      0.00

#2345678901234567890123456789012345678901234567890      Ruler
```

- The first line specifies:
  - The calculation code (`ae` here stands for "all-electron").
  - A title for the job (here `Si ground state all-electron`).

  (format 3x,a2,a50)
- Second line:
  - Chemical symbol of the nucleus (here `Si`, obviously)
  - Exchange-correlation type. Here, `ca` stands for Ceperley-Alder. The "best" LDA choice should be `ca`. It is also possible to use a gradient-corrected functional: `pb` indicates use of the PBE scheme by Perdew, Burke, and Ernzerhof (PRL 77, 3865 (1996)). The exchange-correlation energy and potential are computed using a modified Soler-Balbás package.

- The character `r` next to `ca` is a flag to perform the calculation relativistically, that is, solving the Dirac equation instead of the Schrodinger equation. The full range of options is:
  - `s` : Spin-polarized calculation, non-relativistic.
  - `r`: Relativistic calculation, obviously polarized.
  - (blank) : Non-polarized (spin ignored), non-relativistic calculation.

  (format 3x,a2,3x,a2,a1,2x)

- Third line. Its use is somewhat esoteric and for most calculations it may contain just zeros (six numbers) as the code will use the default values.

  The first number is the nuclear charge, Z. You can invent elements with fractional charge. If it is zero the program will get it from the chemical symbol of the previous line. The second and third number are there for historical reasons. Their values are never used. The fourth to sixth numbers specify the integration mesh. Again if they are zero the code uses default values. The fourth number is the maximum range of the radius values $r_{\max}$. The sixth number B controls the increment between consecutive grid points. $r(n+1) \sim (1+1/B)r(n)$, in other words it takes about B points to increase the radius in the mesh by about e. The fifth number A controls the first non-zero point in the mesh. $r(1) = 0$, $r(2) \simeq exp(-A)/(Z * B)$.

  The full specification of the quasi-logarithmic mesh is $a = exp(-A)/Z$, $b = 1/B$, $r(i) = a * (exp(b * (i-1)) - 1)$, $r(i) < r_{\max}$.

The rest of the file is devoted to the specification of the electronic configuration:

- Fourth line:
  Number of core and valence orbitals. For example, for Si, we have $1s$, $2s$, and $2p$ in the core (a total of 3 orbitals), and $3s$ and $3p$ in the valence complex (2 orbitals).

  (format 2i5)

- Fifth, sixth... lines: (there is one line for each valence orbital)
  - `n` (principal quantum number)
  - `l` (angular momentum quantum number)
  - Occupation of the orbital in electrons.

  (format 2i5,3f10.3)

  (There are two f input descriptors to allow the input of "up" and "down" occupations in spin-polarized calculations (see example below)). There is a third float point number that can fix the eigenvalue of the orbital. It has not been used in years, Forget it exists.

  The valence orbitals can be given in any order. The same orbital cannot be given twice, nor can a valence orbital be one of the core orbitals (the program stops in those cases). Two valence orbitals with the same angular momentum and different principal quantum numbers are allowed.

The different treatment of core and valence orbitals in the input for an all-electron calculation is purely cosmetic. The program "knows" how to fill the internal orbitals in the right order, so it is only necessary to give their number. That is handy for heavy atoms... Overzealous users might want to check the output to make sure that the core orbitals are indeed correctly treated.

For compatibility with the old code, the file should end with an empty line. The new code doesn't care about it.

For a **pseudopotential test calculation**, the format is exactly the same, except that the job code is `pt` instead of `ae`.

For a **pseudopotential generation run**, in addition to the electronic configuration chosen for the generation of the pseudopotentials (which is input in the same manner as above), one has to specify the "flavor" (generation scheme) and the set of core radii r_c for the construction of the pseudowavefunction. Here is an example for Si using the Troullier-Martins scheme:

```
  pg     Si     Pseudopotential
       tm2     2.00
n=Si c=ca
      0.0       0.0       0.0       0.0       0.0       0.0
   3    3
   3    0     2.00      0.00
   3    1     0.50      0.00
   3    2     0.50      0.00
     1.12      1.35      1.17       0.0       0.0       0.0

#2345678901234567890123456789012345678901234567890123456789012345678901234567890   Ruler
```

Apart from the `pg` (pseudopotential generation) job code in the first line, there are two extra lines:

- Second line:

  Flavor and radius at which to compute logarithmic derivatives for test purposes.

  The flavor can be one of : tm2 Improved Troullier-Martins

  (format 8x, a3, f9.3)

- The last line (before the blank line) specifies, as in SIESTA and the old versions of the code:

  - The values of the r_c in atomic units (bohrs) for the s, p, d, and f orbitals. They are given by angular momentum, whatever the order of the valence orbitals in the previous lines (it is a good practice to input the valence orbitals in the order of increasing angular momentum, so that there is no possible confusion).
    (format 4f10.5)
  - Two extra fields (2f10.5) which are relevant only if partial core corrections are used (see Sect 4.3). If they are zero or blank default values are used.
  - In the most recent versions of the code an extra field (f10.5) is used for the definition of the matching radius of the partial core kinetic energy density.

  The fields are read in fixed columns: the core radii in columns 1–40, `rcore_flag` in columns 41–50, r_{pc} in 51–60 and the kinetic energy density flag in 61–70. Any of the last three can be left blank. Versions of the code from 2021 to 2026 wrote these three numbers in columns 51–80 (with columns 41–50 blank); such files should be edited to move them ten columns to the left.

In the `tm2` example above, only s ,p, and d r_c's are given. Here is an example for Silicon in which we are only interested in the s and p channels for our pseudopotential:

```
   pg     Si     s and p only
        tm2     2.00
 n=Si c=ca
       0.0       0.0       0.0       0.0       0.0       0.0
    3    2
    3    0     2.00      0.00
    3    1     2.00      0.00
      1.80      1.80      0.00       0.0       0.0       0.0

#2345678901234567890123456789012345678901234567890123456789012345678901234567890   Ruler
```

The new version of the code has an extension for the case a g orbital (or higher angular momentum) is present in the valence. In that case the line has one r_c field for each angular momentum up to the largest one in the valence, for example five fields (format 5f10.5) for a g orbital, and the core correction fields follow immediately after them (columns 51–80 in that example). Pseudopotentials are generated up to g ($l = 4$).

This completes the discussion of the more common features of the input file.

5 A primer on ab-initio pseudopotentials

We will not review here the background of density functional pseudopotentials. But, in this case more than ever, there is a lot to be gained from reading the original literature. Here are a few basic references:

- Some seminal papers on ab-initio pseudopotentials:

  Kerker, J. Phys. C 13, L189-94 (1980)
  Hamann, Schluter, Chiang, Phys. Rev. Lett. 43, 1494 (1979)

- More on HSC scheme:

  Bachelet, Schluter, Phys. Rev. B 25, 2103 (1982)
  Bachelet, Hamann, Schluter, Phys. Rev. B 26, 4199 (1982)

- Troullier-Martins elaboration of Kerker method:

  Troullier, Martins, Phys. Rev. B 43, 1993 (1991)

- Core corrections:

  Louie, Froyen, Cohen, Phys. Rev. B 26, 1738 (1982)

- The full picture of plane-wave pseudopotential ab-initio calculations:

  W. E. Pickett, "Pseudopotential Methods in Condensed Matter Applications", Computer Physics Reports 9, 115 (1989)

  M. C. Payne, M. P. Teter, D. C. Allan, T. A. Arias and J. D. Joannopoulos, "Iterative minimization techniques for ab initio total-energy calculations: molecular dynamics and conjugate gradients", Rev. Mod. Phys. 64, 1045, (1992)

Also, the book by Richard Martin "Electronic Structure: Basic Theory and Practical Methods" (Cambridge University Press) has a chapter on pseudopotentials.

- Use in SIESTA:

  J.M. Soler, E. Artacho, J.D. Gale, A. Garcia, J. Junquera, P. Ordejon, D. Sanchez-Portal, "The SIESTA method for ab initio O(N) materials simulation", Jour. Phys.: Condens. Matter, 14, 2745-2779 (2002).